

# INDIAN INSTITUTE OF TECHNOLOGY ROPAR



## Line Follower Robot Using Arduino

### 1. Group Members

| Name             | Entry Number   | Mobile Number |
|------------------|----------------|---------------|
| Ruchi Yadav      | (2024EEB1241)  | 7317728050    |
| Saksham Kumar    | (2024EEB1242)  | 7701985688    |
| Samardeep Singh  | (2024EEB1243)  | 7851854692    |
| Sanket Mamadapur | (2024EEB1244)  | 7411053197    |
| Satyabalaji      | (2024EEB1245)  | 9944111837    |
| Sharad Chand     | ( 2024EEB1246) | 8587838014    |

### 2. Materials Required



Arduino Uno



IR Sensor (2)

- ✓ 2WD car kit
- ✓ L298N motor driver module
- ✓ 7-12 V DC Battery (in our case lipo 2s battery)
- ✓ Jumper wires
- ✓ Glue gun
- ✓ Double sided tape
- ✓ Black Electrical Tape



## 3. Theory

### Line Detection:

The line sensors continuously read the surface below the robot. Each sensor detects whether it is over the line or the background.

Typically, the sensors are arranged in an array at the front of the robot. The number and placement of sensors can vary based on design and complexity.

### Sensor Input Processing:

The sensor readings are sent to the Arduino. The Arduino reads these inputs to determine the robot's position relative to the line. Common sensor configurations include a single sensor, two sensors, or multiple sensors in a line.

### Decision Making:

The Arduino processes the sensor data using predefined algorithms to make decisions about the robot's movement. For example:

**Single Sensor:** The robot might be programmed to move forward if the sensor detects the line, and turn if it does not.

**Two Sensors:** The robot can follow a line more precisely by adjusting its direction based on which sensor detects the line. For instance, if the left sensor detects the line, the robot might turn left to correct its course.

**Multiple Sensors:** More complex algorithms can be used to handle curves and intersections by interpreting the data from multiple sensors.

### **Motor Control:**

Based on the decision-making process, the Arduino sends signals to the motor driver to adjust the speed and direction of the motors. This allows the robot to follow the line accurately. For example, if the robot starts to drift off the line, the Arduino may command the motors to steer the robot back towards the line.

### **Feedback Loop:**

The system continuously updates based on real-time sensor data. This feedback loop allows the robot to make constant adjustments to stay on track.

## **4. Algorithms**

**Basic Line Following:** Simple algorithms such as "Follow the Line" or "Proportional Control" can be used. In a basic algorithm, if the sensor on one side of the robot detects the line, it will correct the robot's direction by turning toward the detected line.

## **5. Procedure**

### **1. Assemble the Chassis:**

**Mount the Motors:** Attach the DC motors to the chassis. Securely fix the wheels to the motors.

**Install the IR Sensors:** Attach the IR line sensors to the front of the robot. Position them so that they are close to the ground for better line detection.

### **2. Wiring:**

**Connect IR Sensors to Arduino:**

Connect the sensor output pins to the analog or digital input pins on the Arduino (depending on sensor type).

Connect the sensor power (Vcc) and ground (GND) to the Arduino's 5V and GND pins.

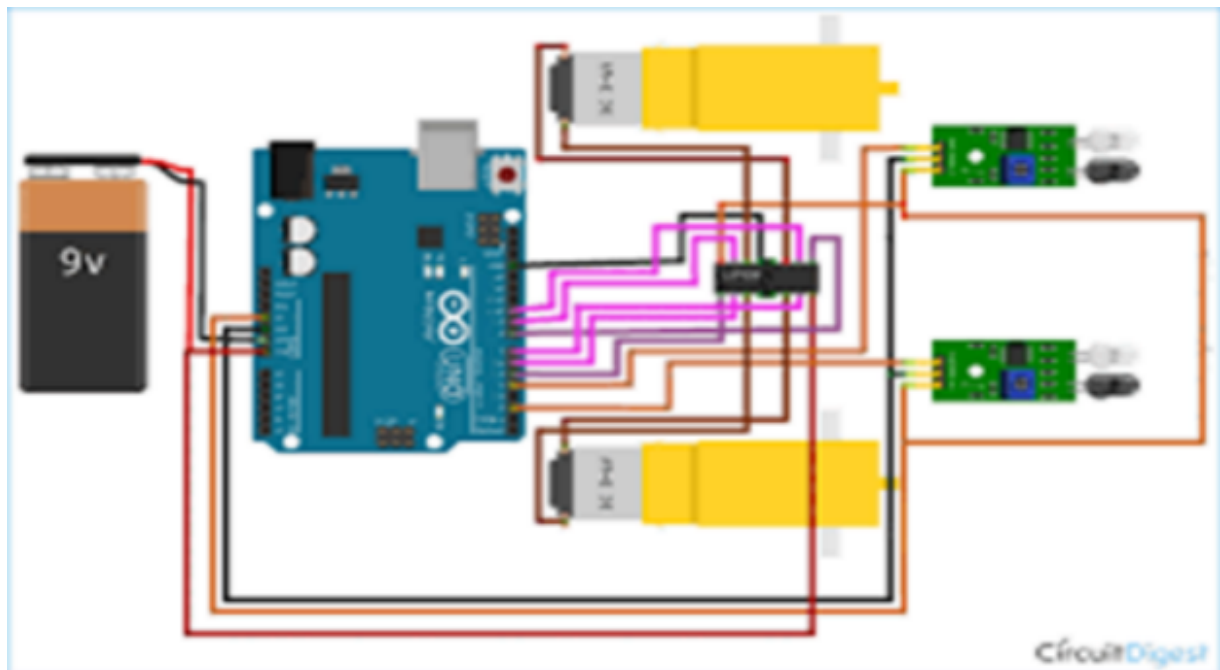
### Connect Motors to Motor Driver:

Wire the motor terminals to the motor outputs of the motor driver module. Connect the motor driver's input pins to digital output pins on the Arduino (for motor control).

Connect the motor driver's power pins to the battery pack (and ensure the motor driver's ground is connected to the Arduino's ground).

### 3. Programming the Arduino:

Install Arduino IDE: Download and install the Arduino IDE from the Arduino website.



Write the Code: Use the code as a basic template for a line-following robot. Here we wrote a code for 2 motors and 2 IR sensors.

Upload the Code: Connect Arduino to computer using a USB cable, select the correct board and port in the Arduino IDE, and upload the code to the Arduino.

### 4. Testing and Calibration:

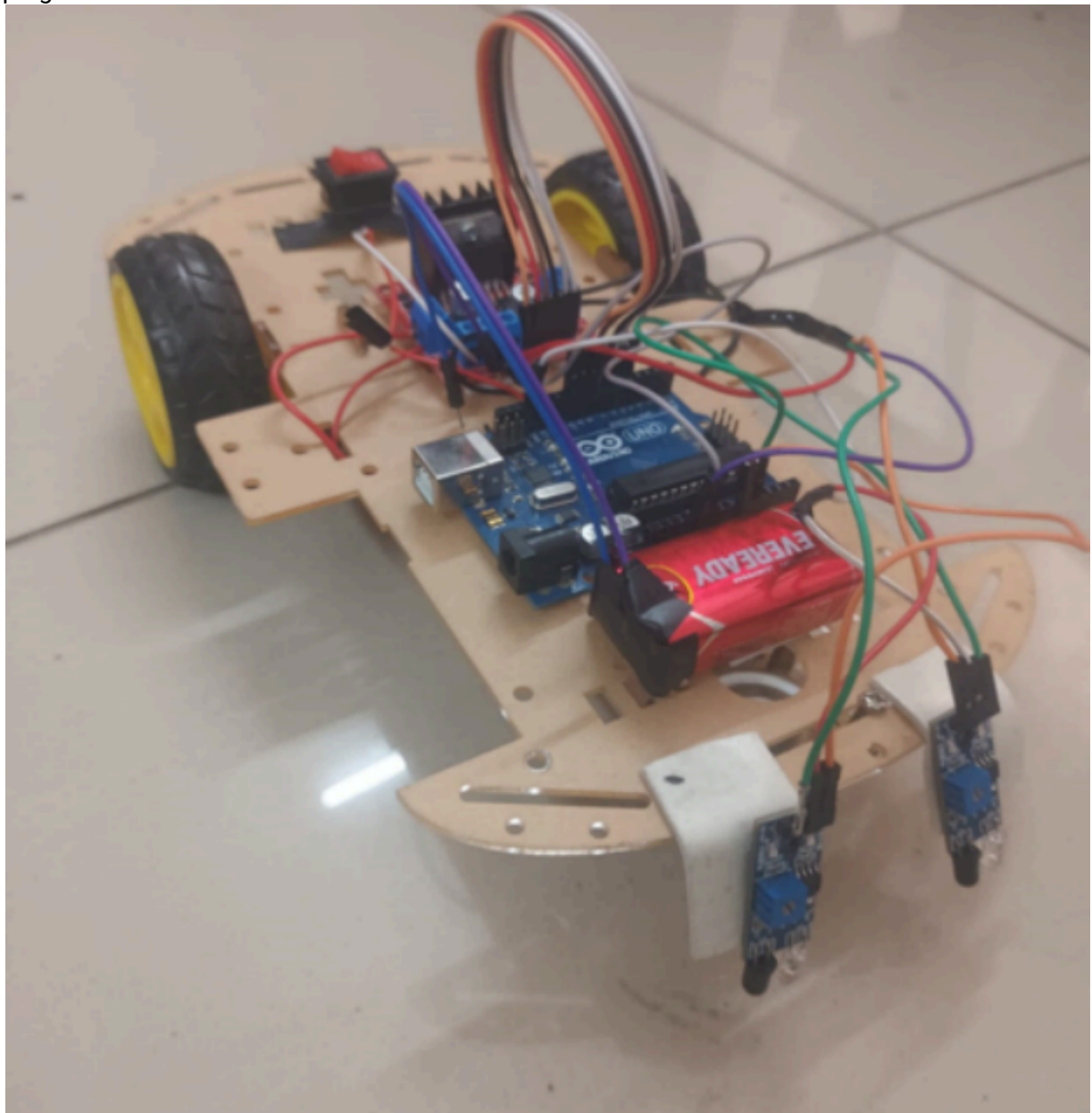
Test the Robot: Place the robot on a track with a line and observe its behavior. Adjust the sensor threshold and motor control logic as needed.

Calibrate Sensors: If the robot does not follow the line as expected, adjust the placement of sensors and change the threshold values in the code.

## 5. Optimization:

**Improve Line Following:** Implement more sophisticated algorithms (like PID control) for better performance and smoother operation.

**Enhance Reliability:** Test under various conditions and fine-tune the robot's parameters for perfect working of the robot. By following these steps, we'll be able to build and program a functional line follower robot



## 6. Results and Conclusion

The line follower robot project highlights the ingenuity of Arduino in creating autonomous systems. By combining sensors, motors, and smart algorithms, you've built a robot that adeptly navigates paths. This hands-on project not only demonstrates the thrill of robotics but also

sharpens our skills in technology and problem-solving. They can be used in various fields, such as manufacturing, agriculture, and education but also if we add extra features to improve performance increases cost, complexity, and energy consumption.

## 7. Contribution

### **Ruchi Yadav (2024EEB1241)**

- Skillfully tested the project
- Contributed in assembling the circuit

### **Saksham Kumar (2024EEB1242)**

- Devised the project idea
- Contributed in assembling the project and designing the template

### **Samardeep Singh (2024EEB1243)**

- Designed the working model and circuit of the project
- Theorized the project on paper

### **Satyabalaji (2024EEB1244)**

- Simulated the robot control system
- Contributed in assembling the project

### **Sanket Mamadapur (2024EEB1245)**

- Researched the basic workings of the project
- Programmed the code for robot

### **Sharad Chand (2024EEB1246)**

- Fabricated the model of robot
- Researched for the circuital design